

iHelp — Basic Scale

1. The Scale Envelope This MVP Targets

iHelp is designed for a **single pilot area with tens of active users and hundreds of requests** (product spec §14). Every mechanism below is chosen to be *correct and simple* inside that envelope, with a named successor for when the envelope breaks. That is the honest trade this document defends: no premature infrastructure, no silent cliffs — each limit is known, bounded, and monitored by an explicit signal.

2. What Happens at Tens → Hundreds of Users

Load source	At 10s of users	At 100s of users	First thing to break
Feed (/requests)	Trivial: one indexed SELECT, ≤ 200 rows, in-memory sort	Still fine: the fetch is capped at 200 rows regardless of table size	Requests beyond the newest 200 stop appearing in the feed (see §5)
Request detail	4–6 single-key reads + one bulk signed-URL call	Same — all keyed by <code>request_id</code>	Nothing until offers-per-request grow unusually large
Writes (offers, transitions)	Single-row RPCs with row locks	Row locks serialize only <i>per request</i> — different requests never contend	Many simultaneous offers on one hot request briefly serialize on that request's status update (acceptable at this scale)
Auth/session	Middleware refresh per request	Supabase Auth scales independently of us	—
Admin review queue	Human-bound, not machine-bound	The manual queue is the product's real bottleneck (spec §14) — a staffing problem before a technical one	Queue latency, visible in <code>idx_applications_queue</code>
Storage	≤ 5 MB/object, ≤ 5 photos/request	Linear growth, CDN-served signed URLs	Orphaned objects accumulate (§7)

Postgres itself (Supabase's smallest tier) handles this envelope with orders of magnitude to spare; the interesting limits are all in *our* choices, listed in §7.

3. Heavy Queries and the Indexes That Serve Them

Every recurring query has a matching index (defined in the schema migrations); none of them require a table scan at any realistic size:

Query (page)	Shape	Index
Feed	<code>status in (open,has_offers) and not is_hidden order by created_at desc limit 200</code>	<code>idx_requests_browse (status, is_hidden, created_at desc)</code>
My requests	<code>requester_id = ? order by created_at desc</code>	<code>idx_requests_owner</code>
Offers on a request (detail)	<code>request_id = ? (offers visible to the caller under RLS)</code>	<code>idx_offers_request</code>
My offers	<code>helper_id = ? order by created_at desc</code>	<code>idx_offers_helper</code>
Photos of a request	<code>request_id = ? order by position</code>	<code>idx_photos_request</code>
Helper rating aggregate	<code>helper_id = ? on ratings / the view</code>	<code>idx_ratings_helper</code>
Admin queue	<code>status='pending' order by created_at</code>	<code>idx_applications_queue</code> (partial on pending)

The two partial indexes are deliberate: they stay small (only live rows) and double as documentation of the hot predicates.

The one query that cannot be pushed to the database today is distance ordering: Haversine is computed in TypeScript (`lib/geo.ts`), so the DB cannot `ORDER BY distance`. The mitigation is the cap (§5); the successor is a SQL Haversine expression — no PostGIS needed — computed over an indexed bounding-box prefilter.

4. Avoiding Over-Fetching

- **Column lists, not `*`, on every list query** (feed, offers, my-*, admin, ratings) — the feed ships 7 columns per row, not the row. The single-row request-detail read uses `*` deliberately: one row, most columns render.
- **Reads are Server Components**: the browser receives rendered HTML, never raw rows. Two coordinate sets, two rules (architecture §8.1): a *viewer's own home* coordinates (`profiles_private`) never leave the server — they feed the Haversine sort in the RSC and only formatted distances reach the browser; *request* coordinates are deliberately shipped to the client as map pins (the feed map needs points to plot), which is the accepted, documented exception — requests carry a location the requester confirms at publish time, and it is already signed-in-readable under RLS (spec §9.3).
- **Photos load per page, not per table**: only the first photo of the 12 visible cards is fetched and signed (one bulk `createSignedUrls` call), not every photo of every request.
- **Rating aggregates are computed per page** over only the helpers actually rendered (`in (helperIds)`), not globally.

5. Pagination — Where It Is Real and Where It Is Honest

- **Feed:** fetch capped at **200 newest matching rows** → Haversine-sort in memory → serve **12 per page** via URL param (`?page=N`). Back/forward and sharing work because the URL is the state. The honest limitation: a request older than the newest 200 in its filter drops out of the feed even if it is close by. At MVP scale (hundreds of *total* requests, most not open) the cap is far above reality; the moment it becomes visible, the successor is **DB-side distance ordering + keyset pagination** (order by a SQL Haversine expression, paginate by `(distance, id)` cursor) — a migration-sized change isolated to one query, which is exactly why the cap was acceptable now.
- **My requests / my offers / admin lists:** unpaginated by design — they are per-user (or per-admin) lists measured in dozens; adding pagination there would be UI complexity with no load benefit. The admin moderation list is capped at the newest 50.

6. Client/Server Separation Under Load

The separation was chosen for security, but it is also the scaling story:

- **All reads happen server-side** (RSC) close to the DB; the client never polls, never holds a cache to invalidate, and re-fetches only on navigation or after its own mutation (`revalidatePath`).
- **All writes are single round-trips** (Server Action → one RPC/one statement).
- **The only browser↔Supabase traffic is file upload** — deliberately, so multi-MB payloads never transit the Next.js server (architecture §5). Uploads scale with Supabase Storage, not with our functions.
- **The one third-party runtime dependency is OpenStreetMap raster tiles** — display-only, keyless, free; a tile-server outage degrades to a blank map square, never breaks a flow. The browser fetches tiles directly from the OSM tile CDN, so map traffic adds zero load to our servers or the DB.
- Vercel serverless concurrency scales horizontally per request; nothing in the app holds cross-request state, so there is no coordination cost.

7. Current Limits (Known, Bounded, Accepted)

#	Limit	Bound / signal	Accepted because
1	Feed cap 200 before distance sort	Feed misses older open requests; signal: open-request count approaching 200	MVP scale is far below it; successor isolated to one query
2	Rating aggregates computed on read	$O(\text{ratings-per-helper})$ per page	Correctness free of update anomalies; denormalized counters (trigger-maintained) are the successor when profiles get hot
3	Signed URLs generated per page view (request photos, verification docs)	One storage API call per page	1-hour expiry amortizes browsing; a CDN-cached public bucket is the successor if photo traffic dominates — the public <code>avatars</code> bucket already uses that pattern, so profile pictures render with no per-view signing cost
4	Orphaned storage objects (upload-then-abandon)	Growth $\propto$ abandoned forms; bounded by 5 MB $\times$ 5	Invisible to users; periodic sweep (list objects, anti-join <code>request_photos / doc_path</code>) is a 20-line admin script
5	Manual admin review queue	Human latency; signal: pending-queue age	The trust model <i>requires</i> a human in the MVP (spec §4.1); semi-automation (SMS OTP) is the roadmap
6	No push/realtime — freshness is navigation-time	Users refresh to see new offers	Product decision (spec §10); Supabase Realtime is a drop-in successor on the offers table
7	Single region (Vercel + Supabase)	Latency for far users	Audience is one country; irrelevant at MVP
8	No rate limiting on RPCs	Abuse ceiling = per-user data caps (one active offer per request, one open application per kind, 5 photos $\leq$ 5 MB)	The DB constraints already bound the damage; see security doc §9 for the roadmap

8. What a Larger-Scale Version Would Change (priority order)

- DB-side distance + keyset pagination** (removes limit #1) — SQL Haversine with a lat/lng bounding-box prefilter; no PostGIS required until city-scale becomes country-scale.
- Trigger-maintained rating aggregates** on `profiles` (removes #2) — the write path already runs through one RPC, so the counter update is one line in `submit_rating`.
- Supabase Realtime on `offers / help_requests`** (removes #6) — the schema is already event-shaped (status enums, timestamps).
- Storage lifecycle job** (removes #4) and image resizing (thumbnails for feed cards instead of full photos).
- Rate limiting** at the edge (middleware token bucket per user) and on auth endpoints (Supabase's built-ins configured stricter).
- Admin tooling**: queue SLAs, bulk review, audit log table — the `verification_applications` audit trail already stores the data.

7. **Read scaling** (only if needed): Supabase read replicas; the app's reads are already stateless and per-request, so pointing RSC reads at a replica is configuration, not redesign.

The through-line: every successor above is *additive* — none requires undoing a decision, because the boundaries (RPC write path, server-side reads, URL state) were drawn where the scale seams are.